

Описание подключения двух датчиков SHT75 для
ESP8266 с использованием базовой библиотеки
SHT1x-ESP от Jonathan Osher (www.practicalarduino.com)

Оглавление

Стр.

Глава 1. Два датчика SHT75 на ESP8266	
со статистическим анализом	3
1.1 Необходимые компоненты	3
1.2 Подключение	3
1.3 Алгоритм работы скетча	3
1.4 Скетч для ESP8266	4
1.5 Важные замечания	4
Глава 2. Подключение двух датчиков SHT75 к Arduino UNO . .	5
2.1 Введение	5
2.2 Схема подключения датчиков	5
2.2.1 Назначение выводов SHT75	6
2.2.2 Электрическая схема подключения	6
2.2.3 Важное замечание о подтягивающих резисторах	7
2.2.4 Рекомендации по питанию	7
2.3 Установка библиотеки	7
2.4 Тестовый скетч	7
2.4.1 Полный код программы	8
2.4.2 Алгоритм работы программы	8
2.5 Возможные проблемы и их решения	8
2.6 Заключение	8
Глава 3. Сравнение платформ ESP8266 и Arduino Uno	
для подключения датчиков SHT75	10
3.1 Сравнительная таблица	10
3.2 Рекомендации по выбору	11
3.3 Заключение	11
Приложение А. Листинг программного кода	13

Глава 1. Два датчика SHT75 на ESP8266 со статистическим анализом

1.1 Необходимые компоненты

- Плата ESP8266 (NodeMCU, Wemos D1 mini и аналогичные)
- Два датчика SHT75
- Библиотека **SHT1х-ESP** (автор Jonathan Oker). Установка:
Инструменты → Управление библиотеками → поиск SHT1х-ESP.

1.2 Подключение

Питание строго **3.3 В!** Подтягивающие резисторы не требуются — библиотека активирует встроенную подтяжку портов.

SHT75 №1	ESP8266 Pin	SHT75 №2	ESP8266 Pin
VCC (красный)	3.3V	VCC (красный)	3.3V
GND (чёрный)	GND	GND (чёрный)	GND
DATA (синий)	GPIO14 (D5)	DATA (синий)	GPIO4 (D2)
SCK (жёлтый)	GPIO12 (D6)	SCK (жёлтый)	GPIO5 (D1)

Таблица 1 — Распиновка двух датчиков

1.3 Алгоритм работы скетча

1. Каждые 5 секунд считываются температура и влажность с обоих датчиков.
2. В Serial Monitor выводятся текущие значения и разность между датчиками ($\Delta T = T_1 - T_2$, $\Delta H = H_1 - H_2$).

3. После накопления **SAMPLE_SIZE** (по умолчанию 10) успешных измерений вычисляются и выводятся:
 - Средние значения температуры и влажности для каждого датчика отдельно.
 - Дисперсии (выборочные) температуры и влажности для каждого датчика.
 - Среднее и дисперсия разности показаний $(\Delta T, \Delta H)$.
4. Накопления обнуляются, начинается новая серия.

1.4 Скетч для ESP8266

Основные моменты:

- Задержка `delay(5000)` защищает датчики от самонагрева.
- Константа **SAMPLE_SIZE** задаёт размер выборки для расчёта статистик.
- Формулы для выборочной дисперсии реализованы через накопление сумм и сумм квадратов.

1.5 Важные замечания

- Задержка `delay(5000)` критична для точности. При более частых опросах датчики нагреваются, внося ошибку до $+0.1^\circ\text{C}$.
- Если требуется непрерывное накопление без сброса после каждых **SAMPLE_SIZE** измерений, удалите блок «Сброс» в конце условия.
- При ошибках чтения проверьте контакты. На длинных линиях может потребоваться внешний подтягивающий резистор 10 кОм к 3.3 В на линии DATA (штатного программного pull-up достаточно только для коротких проводов).
- Расчёт дисперсии ведётся по формуле выборочной дисперсии: $s^2 = \frac{1}{n-1} \left(\sum_{i=1}^n x_i^2 - \frac{(\sum_{i=1}^n x_i)^2}{n} \right)$. Это даёт несмещённую оценку.

Глава 2. Подключение двух датчиков SHT75 к Arduino UNO

2.1 Введение

Датчик SHT75 производства компании Sensirion представляет собой высокоточный цифровой датчик температуры и относительной влажности. Данный документ описывает процедуру подключения двух датчиков SHT75 к плате Arduino UNO и предоставляет тестовый скетч для считывания показаний с выводом в последовательный порт.

2.2 Схема подключения датчиков

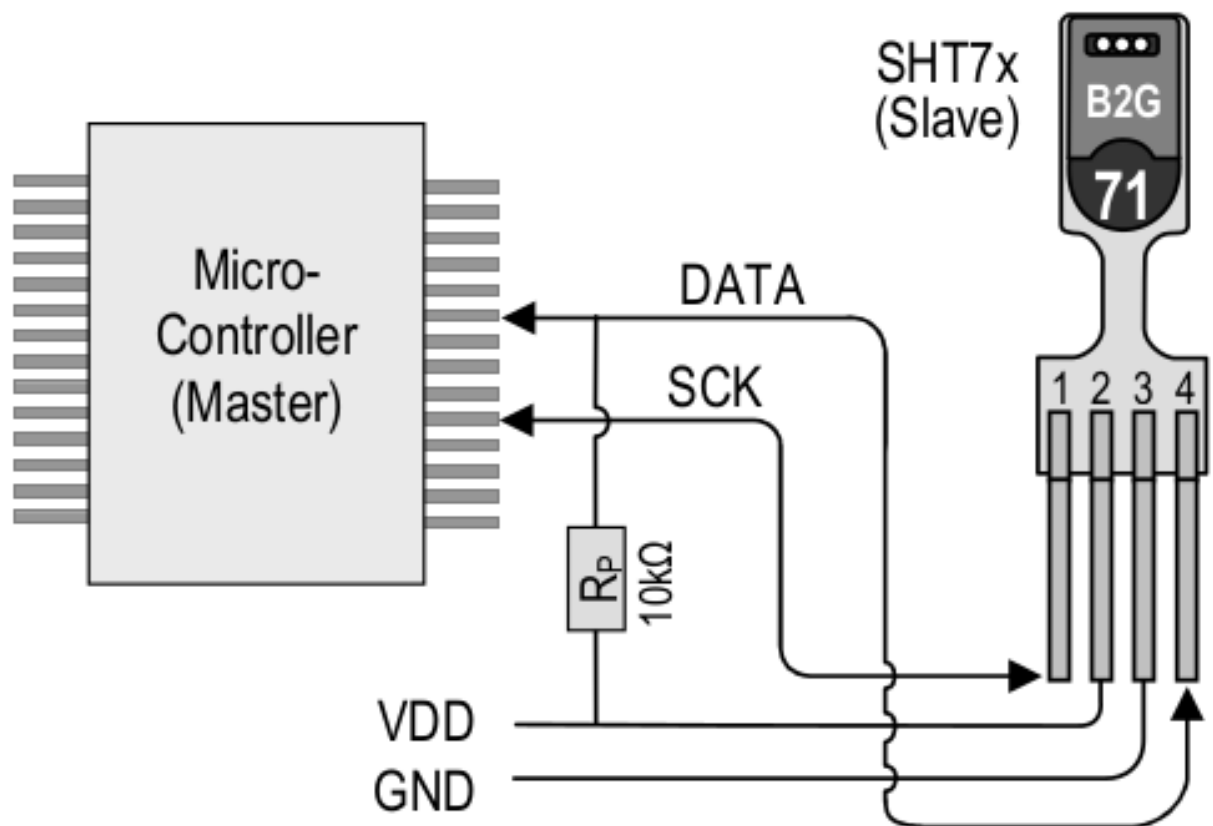


Рисунок 1 — Схема подключения SHT75

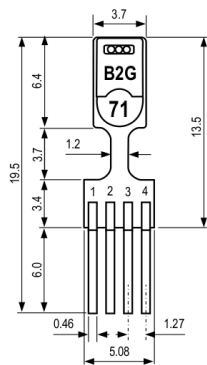


Рисунок 2 — Назначение выводов SHT75

2.2.1 Назначение выводов SHT75

Датчик SHT75 имеет 4 вывода, назначение которых представлено в таблице 2.

Вывод	Обозначение	Назначение
1	SCK	Тактовый сигнал (Clock)
2	VDD	Питание (+5V)
3	GND	Земля
4	DATA	Линия данных

Таблица 2 — Назначение выводов датчика SHT75

2.2.2 Электрическая схема подключения

Подключение двух датчиков SHT75 к Arduino UNO осуществляется согласно таблице 3.

Вывод SHT75	Датчик 1	Датчик 2	Arduino UNO
SCK	Пин 6	Пин 8	Цифровой выход
DATA	Пин 7	Пин 9	Цифровой вход/выход
VDD	+5V	+5V	+5V
GND	GND	GND	GND

Таблица 3 — Подключение выводов датчиков к Arduino UNO

2.2.3 Важное замечание о подтягивающих резисторах

Для корректной работы цифрового интерфейса датчика на линию данных (DATA) каждого датчика необходимо установить подтягивающий резистор (pull-up resistor) номиналом 10 кОм между пином DATA и питанием +5V. Схема подключения резисторов представлена на рисунке ??.

2.2.4 Рекомендации по питанию

- Рекомендуется питать датчики от источника +5V платы Arduino.
- Не рекомендуется питать SHT75 от 3.3V при подключении к 5V-логике Arduino UNO, так как это может вызвать проблемы с согласованием логических уровней.

2.3 Установка библиотеки

Для работы с датчиками SHT75 необходима библиотека **Sensirion**. Процесс установки:

1. Скачайте библиотеку **Sensirion** с официального репозитория GitHub.
2. Переместите папку с библиотекой в директорию **libraries** ваших скетчей Arduino IDE.
3. Перезапустите Arduino IDE для обнаружения новой библиотеки.

После установки библиотеки необходимо добавить директиву в начале скетча:

2.4 Тестовый скетч

Листинг тестового скетча представлен в приложении А.2

2.4.1 Полный код программы

Ниже представлен полный код скетча для считывания температуры и влажности с двух датчиков SHT75 с выводом данных в последовательный порт.

2.4.2 Алгоритм работы программы

1. В функции `setup()` производится инициализация последовательного порта на скорости 9600 бод.
2. Создаются два объекта класса `Sensirion` для каждого датчика с указанием соответствующих пинов.
3. В основном цикле `loop()` для каждого датчика вызывается метод `measure()`, который выполняет полный цикл измерений температуры, влажности и вычисляет точку росы.
4. Полученные данные выводятся в последовательный порт с соответствующими метками.
5. Программа переходит в режим ожидания на 2 секунды перед следующим измерением.

2.5 Возможные проблемы и их решения

2.6 Заключение

Подключение двух датчиков SHT75 к Arduino UNO не представляет сложности при условии соблюдения правильной схемы подключения и использования подтягивающих резисторов. Ключевые моменты:

- Использование уникальных цифровых пинов для каждого датчика;
- Установка подтягивающих резисторов 10 кОм на линиях данных;

Проблема	Возможное решение
Неверные данные	Проверить правильность подключения проводов. Убедиться в наличии подтягивающих резисторов на линии DATA.
Ошибки компиляции	Убедиться, что библиотека <code>Sensirion</code> правильно установлена в директорию <code>libraries</code> .
Отсутствие данных в мониторе порта	Проверить правильность выбора порта и скорости передачи данных
Повреждение датчика при пайке	Использовать паяльник с регулировкой температуры. Минимизировать время контакта жала с выводами датчика.

Таблица 4 — Диагностика и устранение неисправностей

- Использование библиотеки `Sensirion` для создания отдельных объектов-датчиков;
- Подача питания $+5V$ от платы Arduino.

При соблюдении всех рекомендаций система будет стабильно выдавать корректные показания температуры и относительной влажности с обоих датчиков.

Глава 3. Сравнение платформ ESP8266 и Arduino Uno для подключения датчиков SHT75

3.1 Сравнительная таблица

Критерий	ESP8266	Arduino Uno
Совместимые библиотеки	Библиотека SHT1х-ESP, адаптированная для высокочастотных платформ	Библиотеки SHT1х или Sensirion, работающие "из коробки"
Сложность настройки	Средняя. Требуется специализированная библиотека	Низкая. Готовые примеры
Проблемы с таймингами	Высокая частота (80–160 МГц) требует специальной адаптации	Частота 16 МГц соответствует требованиям датчика
Доп. возможности	Встроенный Wi-Fi	Нет сетевых возможностей
Логические уровни	3.3 В	5 В
Поддержка 3.3V калибровки	Библиотека SHT1х-ESP поддерживает 3.3V	Рассчитан на 5V
Требования к питанию	3.3V	5V
Время отклика	Быстрее	Стабильное
Подключение двух датчиков	Поддерживается	Поддерживается

Критерий	ESP8266	Arduino Uno
Стабильность чтения	Высокая	Высокая
Размер кода	Больше	Компактный
Потребление	Выше, есть режимы сна	Предсказуемое

Таблица 5 — Сравнение платформ ESP8266 и Arduino Uno для работы с датчиками SHT75

3.2 Рекомендации по выбору

ESP8266 подходит, если...	Arduino Uno подходит, если...
Требуется передача данных по Wi-Fi	Нет необходимости в сетевой передаче
Есть опыт работы с ESP-платформами	Это первый проект с датчиками
Допустима установка специализированной библиотеки	Нужно быстрое и надежное решение
Питание 3.3V предпочтительно	Есть стабильный источник питания 5V
Важен компактный размер платы	Размер платы не критичен

3.3 Заключение

Arduino Uno обеспечивает простоту и надежность с проверенными библиотеками "из коробки". **ESP8266** требует специализированной библиотеки SHT1x-ESP, но предоставляет встроенный Wi-Fi. Выбор платформы зависит

от задачи: для локальных измерений лучше Arduino Uno, для IoT-проектов — ESP8266.

Приложение А

Листинг программного кода

Листинг А.1 Листинг скетча для ESP8266 двух датчиков SHT75

```

/*
 * Dual SHT75 sensors on ESP8266 with statistical analysis
 * Based on SHT1x-ESP library by Jonathan Oker
 * www.practicalarduino.com
5 */

#include <Arduino.h>
#include <SHT1x-ESP.h>

10 // ===== Пины для датчика 1 =====
#define DATA_PIN_1 14 // GPIO14 (D5)
#define CLOCK_PIN_1 12 // GPIO12 (D6)

// ===== Пины для датчика 2 =====
15 #define DATA_PIN_2 4 // GPIO4 (D2)
#define CLOCK_PIN_2 5 // GPIO5 (D1)

// Объекты датчиков (напряжение питания 3.3V)
SHT1x sht1(DATA_PIN_1, CLOCK_PIN_1, SHT1x::Voltage::DC_3_3v);
20 SHT1x sht2(DATA_PIN_2, CLOCK_PIN_2, SHT1x::Voltage::DC_3_3v);

const int SAMPLE_SIZE = 10; // Размер выборки для статистики

// Переменные накопления
25 int sampleCount = 0;
float sumT1 = 0, sumT2 = 0, sumH1 = 0, sumH2 = 0;
float sumDT = 0, sumDH = 0;
float sumSqT1 = 0, sumSqT2 = 0, sumSqH1 = 0, sumSqH2 = 0;
float sumSqDT = 0, sumSqDH = 0;

30 void setup() {
    Serial.begin(115200);
    Serial.println("Dual SHT75 with statistics");
    delay(1000);
35 }

```

```

void loop() {
    // Чтение двух датчиков
    float t1 = sht1.readTemperatureC();
40    float h1 = sht1.readHumidity();
    float t2 = sht2.readTemperatureC();
    float h2 = sht2.readHumidity();

    // Проверка на корректность значений
45    if (!isnan(t1) && !isnan(h1) && !isnan(t2) && !isnan(h2)) {
        float dT = t1 - t2;
        float dH = h1 - h2;

        // Текущие показания
50        Serial.print("T1=");
        Serial.print(t1, 1);
        Serial.print("C H1=");
        Serial.print(h1, 1);
        Serial.print("% | T2=");
55        Serial.print(t2, 1);
        Serial.print("C H2=");
        Serial.print(h2, 1);
        Serial.print("% | dT=");
        Serial.print(dT, 2);
60        Serial.print("C dH=");
        Serial.print(dH, 2);
        Serial.println("%");

        // Накопление сумм
65        sampleCount++;
        sumT1 += t1;    sumSqT1 += t1 * t1;
        sumT2 += t2;    sumSqT2 += t2 * t2;
        sumH1 += h1;    sumSqH1 += h1 * h1;
        sumH2 += h2;    sumSqH2 += h2 * h2;
70        sumDT += dT;    sumSqDT += dT * dT;
        sumDH += dH;    sumSqDH += dH * dH;

        // Вывод статистики после SAMPLE_SIZE измерений
75        if (sampleCount >= SAMPLE_SIZE) {
            float n = sampleCount;

            float meanT1 = sumT1 / n;
            float meanT2 = sumT2 / n;
            float meanH1 = sumH1 / n;

```

```

80     float meanH2 = sumH2 / n;
        float meanDT = sumDT / n;
        float meanDH = sumDH / n;

        float varT1 = (n > 1) ? (sumSqT1 - sumT1 * sumT1 / n) / (n
- 1) : 0;
85     float varT2 = (n > 1) ? (sumSqT2 - sumT2 * sumT2 / n) / (n
- 1) : 0;
        float varH1 = (n > 1) ? (sumSqH1 - sumH1 * sumH1 / n) / (n
- 1) : 0;
        float varH2 = (n > 1) ? (sumSqH2 - sumH2 * sumH2 / n) / (n
- 1) : 0;
        float varDT = (n > 1) ? (sumSqDT - sumDT * sumDT / n) / (n
- 1) : 0;
        float varDH = (n > 1) ? (sumSqDH - sumDH * sumDH / n) / (n
- 1) : 0;

90     Serial.println("--- Statistics (last " + String(
SAMPLE_SIZE) + " samples) ---");

        Serial.print("T1 : mean=");
        Serial.print(meanT1, 2);
95     Serial.print("C, var=");
        Serial.print(varT1, 4);
        Serial.println("C2");

        Serial.print("T2 : mean=");
        Serial.print(meanT2, 2);
100    Serial.print("C, var=");
        Serial.print(varT2, 4);
        Serial.println("C2");

        Serial.print("H1 : mean=");
        Serial.print(meanH1, 2);
        Serial.print("%, var=");
        Serial.print(varH1, 4);
        Serial.println("%2");
105

        Serial.print("H2 : mean=");
        Serial.print(meanH2, 2);
        Serial.print("%, var=");
        Serial.print(varH2, 4);
        Serial.println("%2");
110

        Serial.print("H2 : mean=");
        Serial.print(meanH2, 2);
        Serial.print("%, var=");
        Serial.print(varH2, 4);
115    Serial.println("%2");

```

```

    Serial.print("dT : mean=");
    Serial.print(meanDT, 4);
    Serial.print("C, var=");
120    Serial.print(varDT, 6);
    Serial.println("C2");

    Serial.print("dH : mean=");
    Serial.print(meanDH, 4);
125    Serial.print("%, var=");
    Serial.print(varDH, 6);
    Serial.println("%2");

    Serial.println("
    -----");

    // Сброс
    sampleCount = 0;
    sumT1 = sumT2 = sumH1 = sumH2 = sumDT = sumDH = 0;
    sumSqT1 = sumSqT2 = sumSqH1 = sumSqH2 = sumSqDT = sumSqDH
= 0;
135    }
} else {
    Serial.println("Sensor read error");
}

140    delay(5000); // Защита от самонагрева датчиков
}

```

Листинг А.2 Листинг тестового скетча для Arduino UNO двух датчиков SHT75

```

#include <Sensirion.h>

// Определение пинов для каждого датчика
const uint8_t SHT75_DATA_PIN_1    = 7;
5 const uint8_t SHT75_SCLK_PIN_1    = 6;

const uint8_t SHT75_DATA_PIN_2    = 9;
const uint8_t SHT75_SCLK_PIN_2    = 8;

10 // Создание объектов для каждого датчика
Sensirion sensor1(SHT75_DATA_PIN_1, SHT75_SCLK_PIN_1);
Sensirion sensor2(SHT75_DATA_PIN_2, SHT75_SCLK_PIN_2);

```

```

void setup() {
15   Serial.begin(9600);
      delay(15); // Ожидание готовности датчика после подачи питания
      Serial.println("--- SHT75 Dual Sensor Test ---");
}

20 void loop() {
      // Переменные для хранения данных от первого датчика
      float temperature1, humidity1, dewpoint1;

      // Переменные для хранения данных от второго датчика
25     float temperature2, humidity2, dewpoint2;

      // --- Измерение данных с первого датчика ---
      // Функция measure() выполняет все три измерения последователь
      но и сохраняет результаты
      sensor1.measure(&temperature1, &humidity1, &dewpoint1);
30

      // --- Измерение данных со второго датчика ---
      sensor2.measure(&temperature2, &humidity2, &dewpoint2);

      // --- Вывод данных в монитор порта ---
35     Serial.println("=== Sensor 1 ===");
      Serial.print("Temperature: ");
      Serial.print(temperature1);
      Serial.println(" *C");
      Serial.print("Humidity: ");
40     Serial.print(humidity1);
      Serial.println(" %");
      Serial.print("Dewpoint: ");
      Serial.print(dewpoint1);
      Serial.println(" *C");
45

      Serial.println("=== Sensor 2 ===");
      Serial.print("Temperature: ");
      Serial.print(temperature2);
      Serial.println(" *C");
50     Serial.print("Humidity: ");
      Serial.print(humidity2);
      Serial.println(" %");
      Serial.print("Dewpoint: ");
      Serial.print(dewpoint2);
55     Serial.println(" *C");

```

```
Serial.println("-----");  
Serial.println();  
60 delay(2000); // Задержка между измерениями  
}
```